



Datenmanagement

Datenintegration

Bachelor Informatik/Softwaretechnik

Prof. Dr.-Ing. Max Zimmermann

Herausforderungen im analyseorientierten Datenmanagement

1. Datenintegration aus verschiedenen heterogenen transaktionsorientierten Systemen (fachlich)
 2. Effiziente Datenspeicherung und -verarbeitung in einem Datenmanagementsystem (technisch)
 3. Aufbereitung der Daten zur betriebswirtschaftlichen Entscheidungsunterstützung (fachlich)
-

Anwendungsszenario: Krankenhaus

– Daten werden verwaltet für

- Patienten
- Ärzte
- Medikamente



Datenbank (Wer produziert die Daten?)

– Arbeit mit den Daten

- Patientendaten aufnehmen
- Behandlungstermine vereinbaren
- Nachbestellen von Medikamenten

- Welche Station war im letzten Jahr wie stark ausgelastet?
- Wie hoch waren die Kosten für Medikamente in den verschiedenen Bereichen?
- Wie lang war die min./max./mittlere Verweildauer bei einer bestimmten Diagnose?



Operative Aufgaben → Tagesgeschäft



Dispositive Aufgaben → Datenanalyse

Anwendungsszenario: Supermarkt

– Daten werden verwaltet für

- Ware
- Lieferanten
- Mitarbeiter

} Datenbank

– Arbeit mit den Daten

- Lagerbestand überwachen
- Neue Waren nachbestellen
- Rechnungen der Lieferanten bezahlen
- Welche Waren sind an welcher Filiale besonders häufig verkauft worden?
- Wie hat sich der Verkauf von bestimmten Warengruppen in den letzten drei Jahren entwickelt?
- Wie hoch ist der Anteil einer Warengruppe am gesamten Umsatz?

} Operative Aufgaben → Tagesgeschäft

} Dispositive Aufgaben → Datenanalyse

Anwendungsszenario: Meteorologie

– Daten werden verwaltet für

- Wetterstationen (Orte)
- Wetterdaten wie Temperatur, Niederschlag, Windstärke, ...

} Datenbank

– Arbeit mit den Daten

- Wetterstationen instand halten
- Wetterdaten protokollieren
- Daten aufbereiten und an die Presse geben

} Operative Aufgaben → Tagesgeschäft

- Wann war letztes Jahr der heißeste Tag?
- Haben sich die Durchschnittstemperaturen je Monat in den letzten Jahren verändert?
- Wie unterscheiden sich die Niederschläge in unterschiedlichen Regionen?

} Dispositive Aufgaben → Datenanalyse

Operative Daten:

- Daten die das tägliche Geschäft der Organisation beschreiben, um den Ist-Zustand zu überwachen.
- Transaktionsorientierte Datenbanken sind optimiert für geringe **Latenz**.

Analytische Daten:

- Daten die für analytische Zwecke verwendet werden, um längerfristige Entscheidungen zu treffen.
- Analyseorientierte Datenbanken sind optimiert für hohen **Durchsatz**.

operational data runs your business while analytical data manages your business

Typen von Informationssystemen

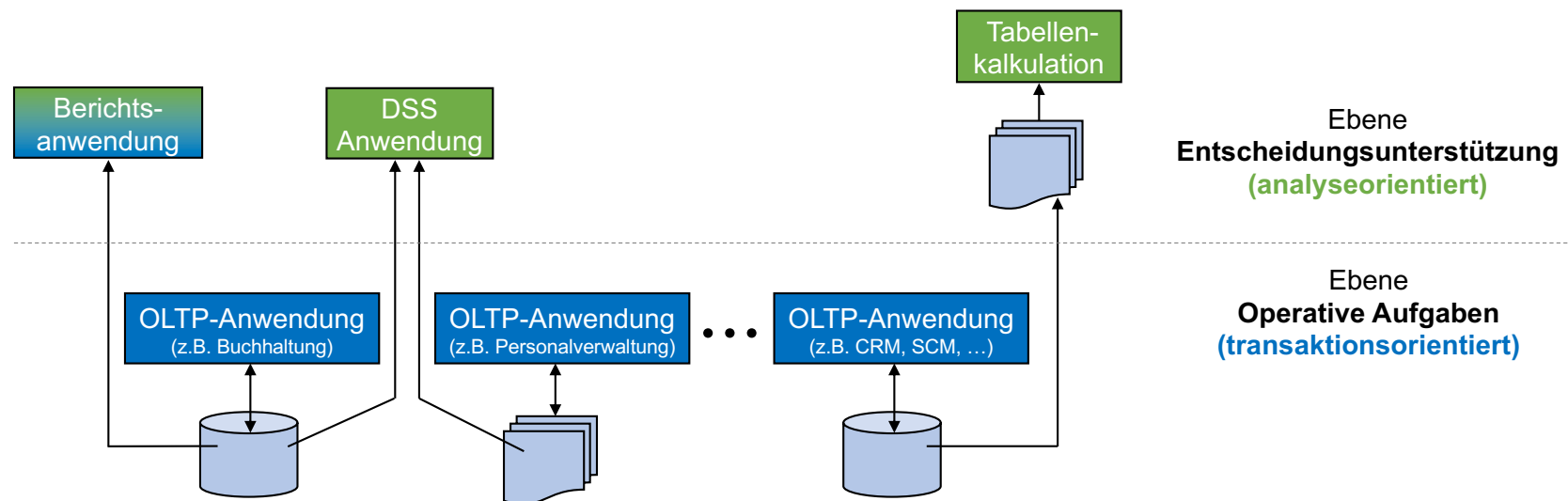
Gruppe/Rolle	Sachbearbeiter	Mittleres Management	Top Management
Aufgabe/Zweck	Operativ	Entscheidungsunterstützung (Taktisch) (Strategisch)	
Beispiel	Buchhaltung	Kosten-Leistungs-Rechnung	Strategisches Controlling
Zeithorizont			
Finanzielle Auswirkungen			
Ausführungsfrequenz			
Systemtyp	OLTP	Decision Support System (DSS) / OLAP	

Online Transactional Processing

Online Analytical Processing

Entscheidungsfindung auf Basis einzelner transaktionsorientierter Systeme ist nicht zielführend

- Schlechte Vergleichbarkeit und Reproduzierbarkeit
- Schlechte Performance
- Mangelnde Flexibilität (z.B. Einbeziehung externer Daten)



Trennung operativer und analytischer Systeme

- Antwortzeitverhalten: Analyse auf transaktionsorientierten Systemen für operative Aufgaben beeinflusst deren Antwortzeitverhalten. Bsp. Direkte Analysen auf ERP Systemen können die Produktion negativ beeinflussen.
- Historisierung: Langfristige Speicherung der Daten erlaubt Zeitreihenanalyse, während viele alte Daten ggf. die operativen Aufgaben behindern.
- Zugriff auf Daten soll unabhängig von konkreten transaktionsorientierten Anwendungen sein (Verfügbarkeit, Integrationsprobleme, Technische Expertise).
- Gewährleistung der Datenqualität zur betriebswirtschaftlichen Entscheidungsunterstützung. Bsp. Erhebung der operativen Daten ist fehlerbehaftet.
- Vereinheitlichung der Datenstrukturen und –formate.

Was ist ein Data Warehouse? – Zitate von CIOs

“One large repository
**for the entire
company**”

“A data warehouse is
a **large database**,
think of multiple
terabytes of data”

“Data warehouses
are **tuned for
analytics**”

“**Single source of truth**
for analysis and reporting”

A Data Warehouse is a **subject-oriented, integrated, non-volatile**, and **time variant** collection of data in support of managements decisions.

Bill Inmon (Father of Data Warehouse), 1996

Fachorientierung (*subject-oriented*)

- Zweck ist Unterstützung **bereichsübergreifender** Auswertungen für unterschiedliche Domänen
- Zentralisierte Bereitstellung der Daten über Geschäftsobjekte (Themen)

Integrierte Datenbasis (*integrated*)

- Verarbeitung von Daten aus mehreren verschiedenen (internen und externen) Datenquellen, z.B. interne Datenbanken oder öffentliche Web-APIs

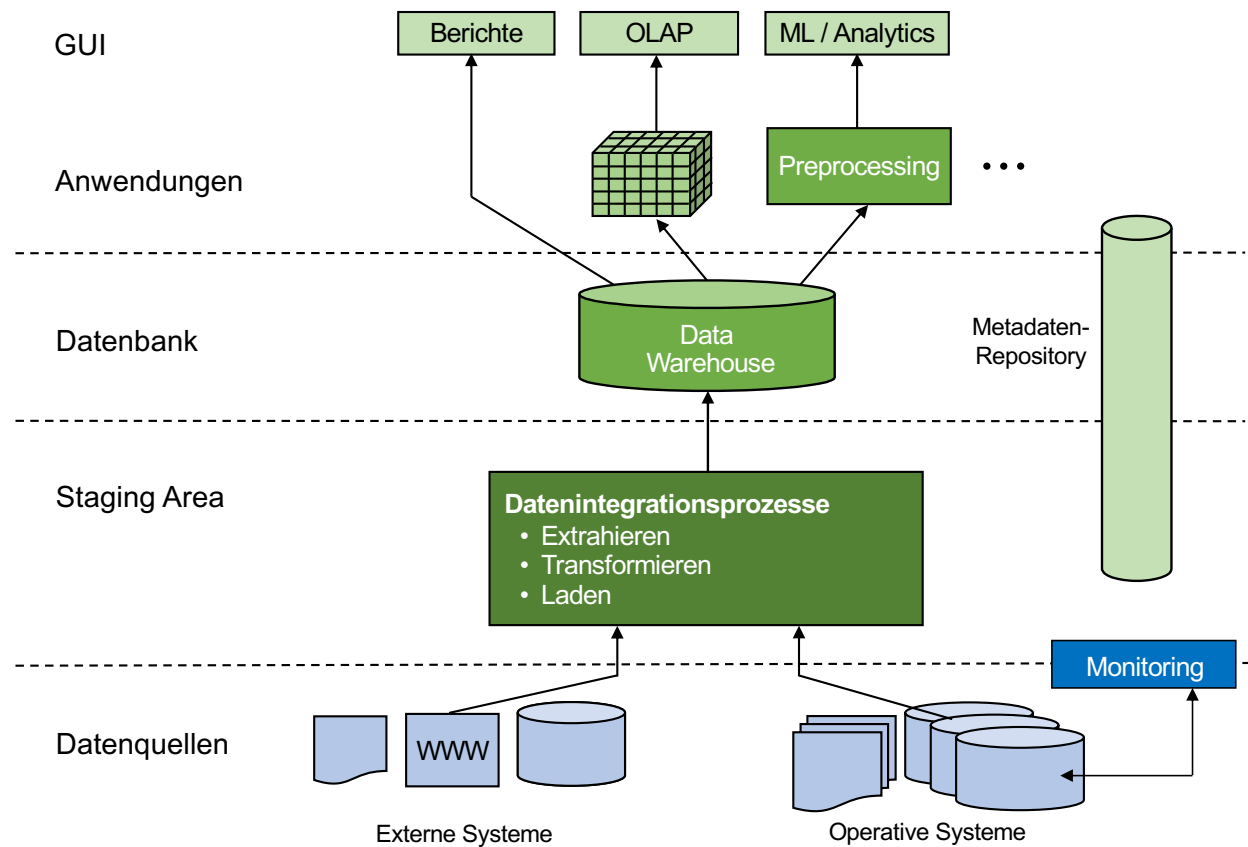
Nicht-flüchtige Datenbasis (*non-volatile*)

- stabile, **persistente** Datenbasis
- Daten im DWH werden i.A. nicht mehr entfernt oder geändert

Zeitbezogene Daten (*time-variant*)

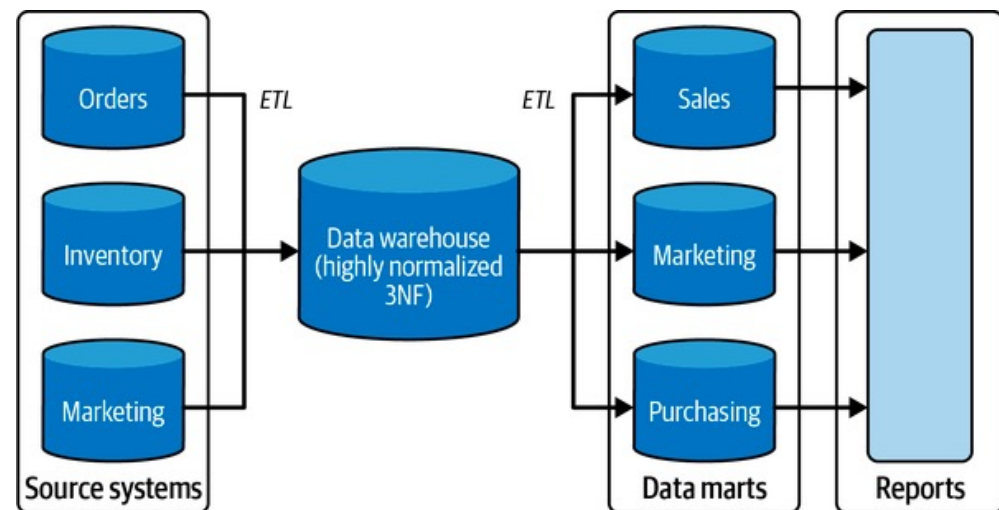
- Vergleich der Daten über Zeit möglich (Zeitreihenanalyse)
- Speicherung über längeren Zeitraum
- Original Daten können über gesamten Zeitraum abgefragt werden

Architektur eines Data Warehouse (DWH)



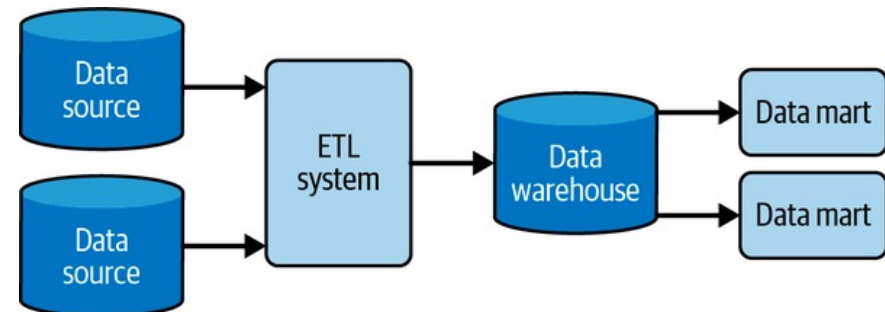
Anforderungen an ein Data Warehouse

- **Dauerhafte Bereitstellung** integrierter und abgeleiteter Daten (Persistenz).
- **Mehrfachverwendbarkeit** der bereitgestellten Daten für beliebige Auswertungen.
- Unterstützung für **individuelle Sichten** (auch adhoc). z.B. bzgl. Zeithorizont, Fachlichkeit und hierarchischer Struktur.
- **Unabhängigkeit** zwischen operativen und analytischen Systemen bzgl. Verfügbarkeit, Last, laufender Änderungen.
- **Erweiterbarkeit**, Integration neuer Datenquellen.
- **Automatisierung** der Analyseprozesse.
- **Eindeutigkeit** in Bezug auf Datenstrukturen, Zugriffsberechtigungen und Analyseprozesse.
- Technische **Ausrichtung am Zweck**
→ Analyse relevanter Daten.

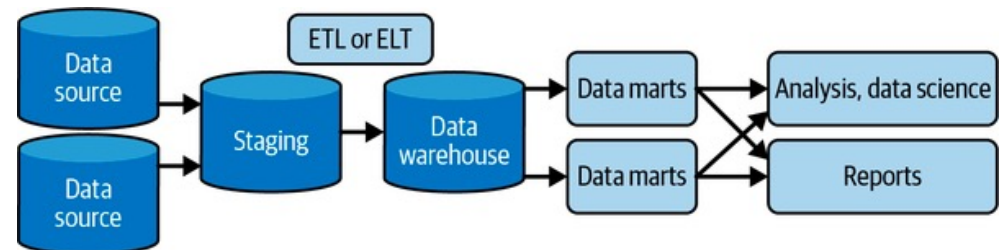


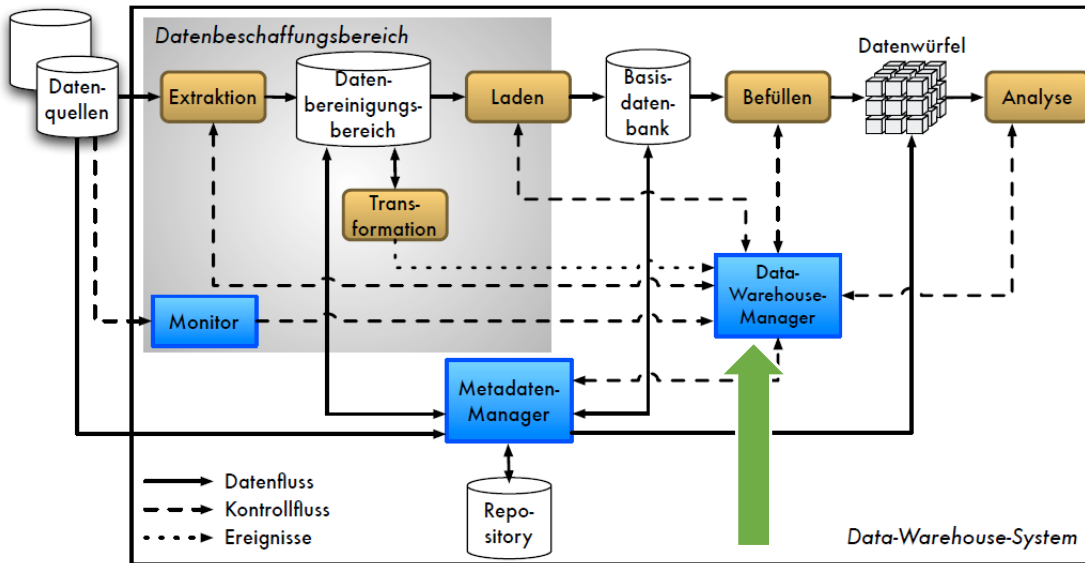
Data Mart:

- Präzisere Teilmengen der Daten für Analysezwecke.
- Geeignet für einzelne Abteilungen, Geschäftseinheiten.
- Bietet die Möglichkeit gezielter Transformationen für einzelne Abteilungen. Bsp. Komplexe Joins, Aggregation von Abteilung Sales.
- Dürfen denormalisiert sein.

**ETL oder ELT:**

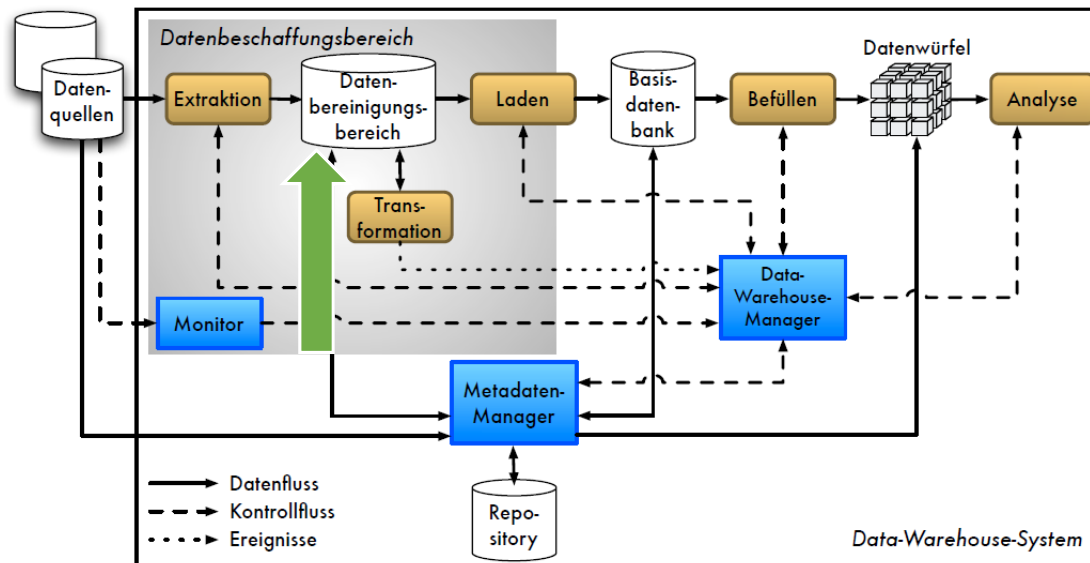
- ELT: Staging Area ist Teil des DWHs.
- Transformation erfolgt im DWH.
- Rechenpower des DWHs für Transformation nutzen.





Data-Warehouse-Manager

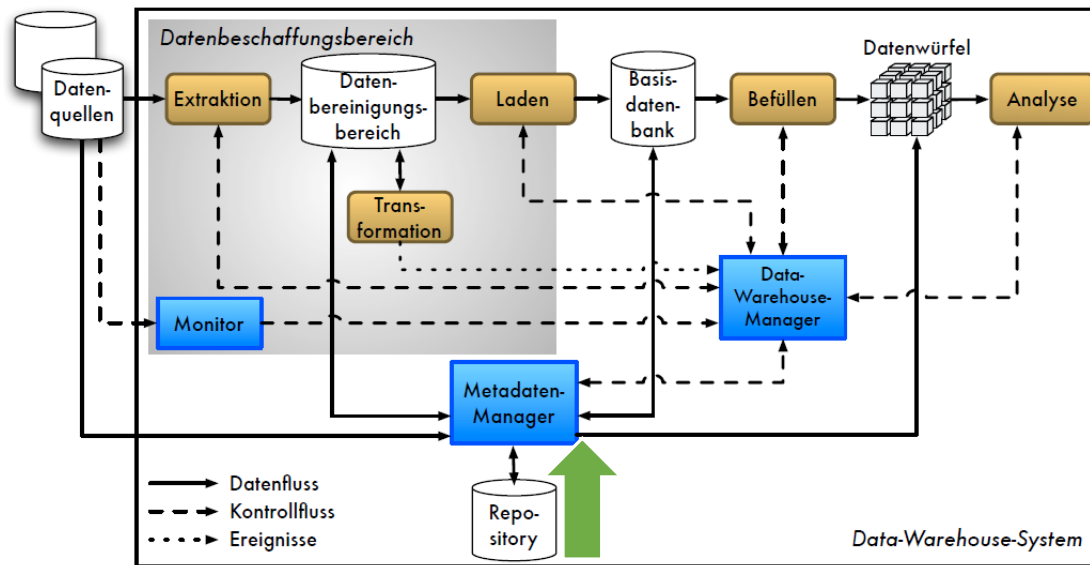
- Initiierung, Steuerung und Überwachung der einzelnen Prozesse, insb. ETL (Extraktion, Transformation, Laden) und Befüllen



Datenbereinigungsbereich (= Staging Area)

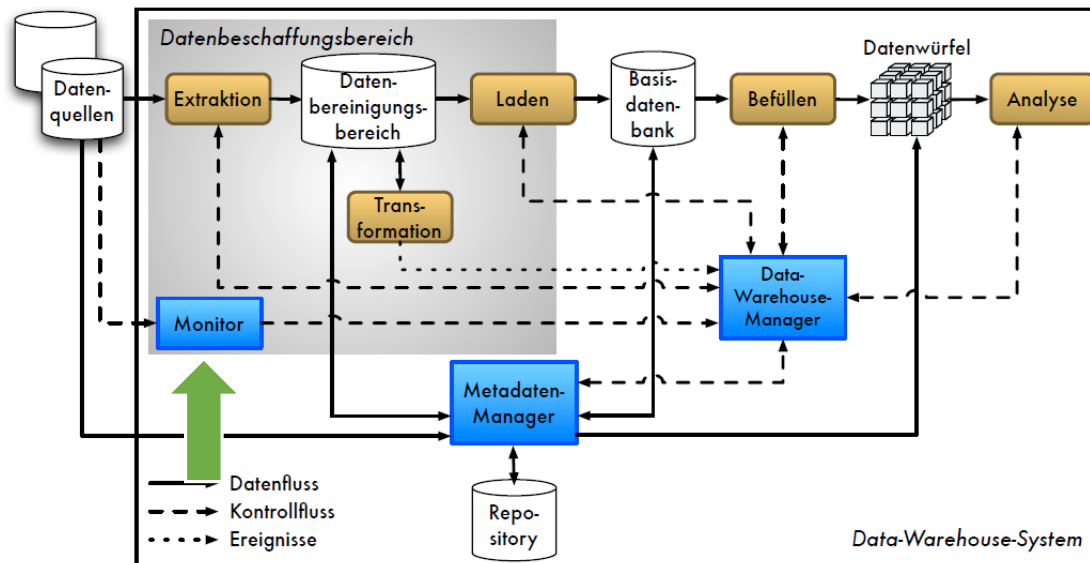
- Temporärer Zwischenspeicher zur Integration
- Ausführung der Transformation auf extrahierten Daten
- Laden der transformierten Daten erst nach erfolgreicher Transformation
- Keine Beeinflussung des Produktionsbetriebs für Datenquellen und DWH
- Keine Übernahme fehlerbehafteter Daten

Köppen, Saake, Sattler: Data Warehouse Technologien, mitp, 2014.



Metadaten-Manager und -Repository (~ Daten Lexikon/Katalog)

- Steuerung der Metadatenverwaltung
- Speicherung der Metadaten
(DB-Schemas, Zugriffsrechte, Prozessbeschreibung z.B. für ETL- und Befüllungsprozesse)
- Versions- und Konfigurationsverwaltung



Monitor

- Entdeckung von Datenmanipulationen in einer Datenquelle
- Auslösen des ETL-Prozesses

Monitoring-Strategien

- Nutzung von DB-Trigger/-Listener → Streaming (Quelle meldet aktiv Änderungen)
- Nutzung von DB-Replikation (Direkter Zugriff auf dedizierte Replikate)
- Analyse von DB-Logs für Transaktionen
- Analyse von Zeitstempeln (Änderungsdatum) je Datensatz
- Periodischer Export aus Datenquelle in Datei → Snapshot-Vergleich

Aktivitäten (Phasen) im Data Warehousing

- Periodische oder kontinuierliche **Überwachung der Datenquellen** auf Änderungen durch **Monitore** (Pull vs. Push).
- **Extraktion** der relevanten Daten in temporären Datenbereinigungsbereich (Staging Area / Landing Zone).
- **Transformation** der Daten im Datenbereinigungsbereich (Bereinigung, Integration).
- **Laden** der Daten in integrierte Basisdatenbank als Grundlage für verschiedene Analysen.
- **Befüllen** aggregierter Datenausschnitte (Data Mart, *OLAP Cube*).
- **Analyse** → OLAP-Operationen, Data Mining, Reporting.

ETL = Extrahieren, Transformieren, Laden

- Kontinuierliche Datenversorgung des DWH
- Sicherung der DWH-Konsistenz bzgl. Datenquellen: Vollständigkeit der Daten
- Phase 1 – von den Quellen zur **Staging Area**
 - Extraktion von Daten aus den Quellen
 - Erstellen/Erkennen von differentiellen Updates (Monitore)
- Phase 2 – von der Staging Area zur Basisdatenbank des DWH
 - Data Cleansing und Tagging
 - Erstellung integrierter Datenbestände
- Effiziente Methoden sind essentiell → **Sperrzeiten minimieren, Quellsystem nicht blockieren.**
- Rigorose Prüfungen sind essentiell → **Datenqualität sichern**

Anforderungen:

- Alte von neuen Daten unterscheiden
- Änderungen in den Daten erkennen
- Nachladen von Daten

Synchrone Benachrichtigung zu Änderungen in Datenquellen

- Quelle propagiert jede Änderung umgehend

Asynchrone Benachrichtigung zu Änderungen in Datenquellen

- Periodisch
 - DWH fragt regelmäßig (täglich/wöchentlich/monatlich) neue Daten ab
 - Quellen erzeugen regelmäßig Extrakte
- Ereignisgesteuert
 - DWH erfragt Änderungen zu einem Ereignis ab (z.B. Monatsabschluss)
 - Quelle informiert DWH alle x Änderungen (z.B. 100 MB neue Daten)
- Anfragegesteuert
 - DWH erfragt Änderungen vor jedem tatsächlichen Zugriff

Snapshots

- Quelle liefert immer **kompletten** Datenbestand
- Herausforderungen: **Änderungen erkennen, Historie korrekt abbilden**
- Differential Snapshot Problem
 - Wiederholtes Transformieren/Laden aller Daten ist ineffizient
 - Duplikate müssen erkannt werden → Algorithmen für inkrementelle Delta-Files werden vorgeschaltet

Logs

- Quelle liefert nur einzelne Änderungen/Deltas
- z.B. DB-Transaktionslogs, anwendungsgesteuertes Logging
- Herausforderung: **Änderungen effizient einspielen**

Aktualität der Daten

Viele (Legacy)-
Quellsysteme
unterstützen nur
diese Methode

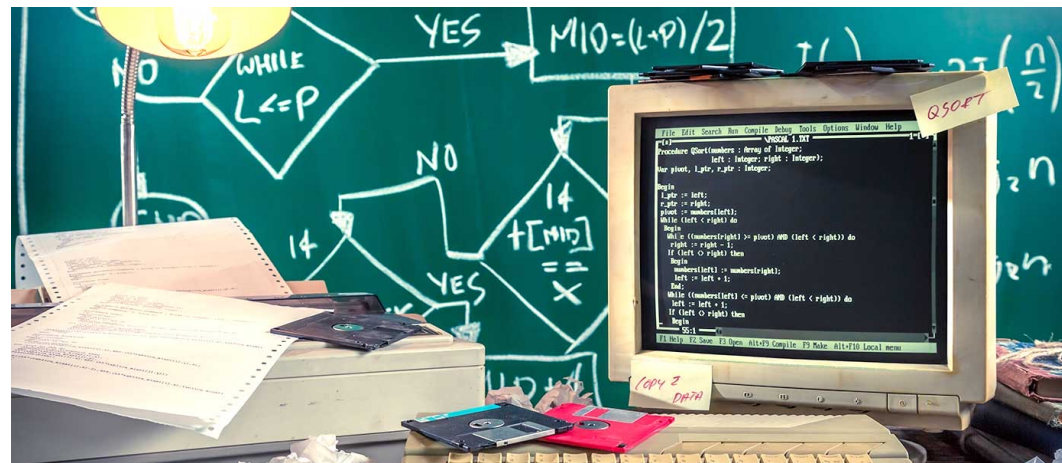
Quelle	Technik	Aktualität DWH	Belastung DWH	Belastung Quelle
... erstellt periodisch Files	Batches, Snapshots	Abhängig von Frequenz	Niedrig	Niedrig
... propagiert jede Änderung	Trigger, Replikationen	Maximal	Hoch	Hoch
... bietet Daten auf Anfrage an	API	Hoch, flexibel	Abhängig von Frequenz	Abhängig von Frequenz

Nur für
Quellsysteme
mit geeigneter
API

Antwortzeitverhalten
des Quellsystems darf
nicht belastet werden

Extraktion aus Legacy-Systemen

- Daten in Non-Standard-Host-Systemen ohne APIs (z.B. PL/I, COBOL, ...)
- Unklare Semantik, keine sprechenden Bezeichner, Doppelbelegung von Feldern
- Fehlende Dokumentation, fehlendes Knowhow, fehlende Motivation
- Herrschaftswissen bei wenigen Legacy-Entwicklern



Differential Snapshot Problem

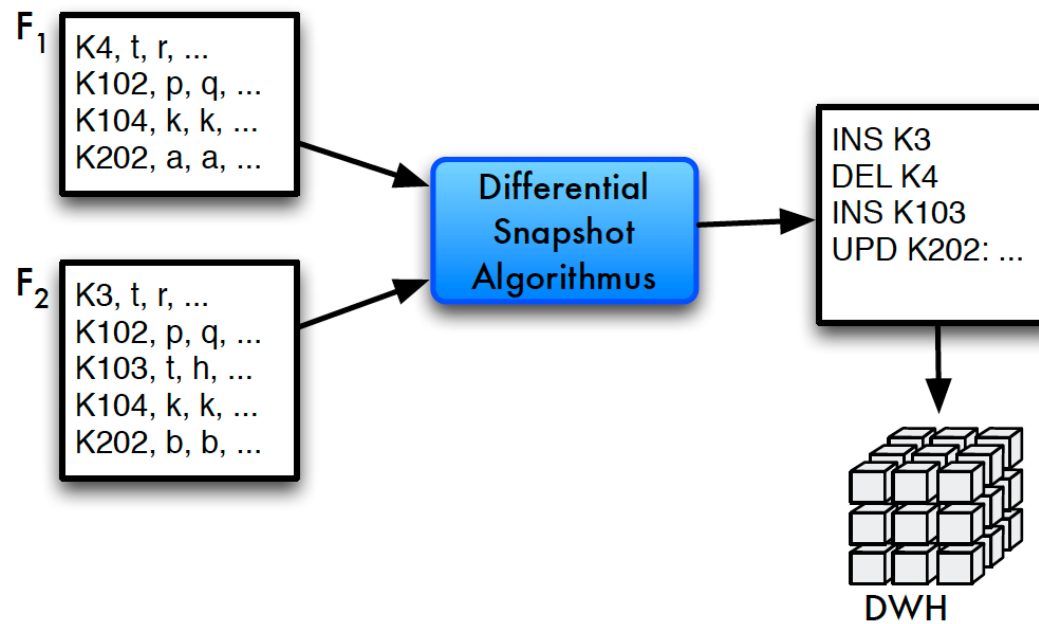
- Viele Quellen liefern immer den vollen Datenbestand
 - Molekularbiologische Datenbanken
 - Kundenlisten, Angestelltenlisten
 - Produktkataloge
- Problem
 - Ständiges Einspielen aller Daten ineffizient
 - Duplikate müssen erkannt werden
- Algorithmen um Delta-Files zu berechnen
- Schwierig bei sehr großen Files

Szenario:

- Quellen liefern Snapshots als File F
 - Ungeordnete Menge von Records (K, A1, . . . , An)
- Gegeben: F1, F2, mit $f1 = |F1|$, $f2 = |F2|$
- Berechne kleinste Menge $O = \{INS, DEL, UPD\}^*$ mit $O(F1) = F2$
- O nicht eindeutig!

$$O1 = \{(INS(X)), \emptyset, (DEL(X))\} \equiv O2 = \{\emptyset, \emptyset, \emptyset\}$$

Szenario



Annahmen

- Berechnung einer konsekutiven Folge von DS
- Files am 1.1.2010, 1.2.2010, 1.3.2010, . . . Kostenmodell
 - Alle Operationen im Hauptspeicher sind umsonst
 - IO zählt mit Anzahl Records: sequenzielles Lesen
- Hauptspeichergröße: M (Records)
- Filegrößen $|F_x| = f_x$ (Records)
- Files i.d.R. **größer** als Hauptspeicher

Naiver Ansatz (Nested Loop):

- Berechnung von O
 - Record R aus F1 lesen
 - F2 sequenziell lesen und mit R vergleichen
 - R nicht in F2 $\rightarrow O := O \cup (\text{DEL}(R))$
 - R in F2 $\rightarrow O := O \cup (\text{UPD}(R))$ / ignorieren
- Problem: **INS** wird nicht gefunden
 - Hilfsstruktur notwendig
 - BitArray mit IDs aus F2 (on-the-fly generieren)
 - R jeweils markieren im Array, abschließender Lauf für **INS**.
- Anzahl IO: $f1 \cdot f2$
- Verbesserungen?
 - Suche in F2 abbrechen, wenn R gefunden
 - jeweils Partition mit Größe M von F1 laden: $\frac{f1}{M} \cdot f2$

Partitionierter Hashing Ansatz:

- Berechnung von O
 - F2 in Partitionen P_i mit $|P_i| = M/2$ hashen
 - Hashfunktion muss garantieren:
$$P_i \cap P_j = \emptyset, \forall i \neq j$$
 - Partitionen sind „Äquivalenzklassen“ bzgl. der Hashfunktion (disjunkte Teilmengen)
 - F1 liegt noch partitioniert vor
 - F1 und F2 wurden mit derselben Hashfunktion partitioniert
 - Jeweils $P_{1,i}$ und $P_{2,i}$ parallel lesen und mischen
- Anzahl IO: $f_1 + f_2$

In der Datenbank mit SQL?

```
INSERT INTO delta
  SELECT 'UPD', ...FROM F1, F2
  WHERE F1.K = F2.K AND F1.W <> F2.W
UNION
  SELECT 'INS', ...FROM F2
  WHERE NOT EXISTS (...)
UNION
  SELECT 'DEL', ...FROM F1
  WHERE NOT EXISTS (...)
```

Auswahl der Datenquellen

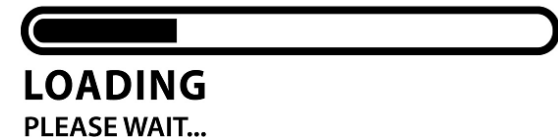
- abhängig vom *Zweck* des Data Warehouse
- von der *Qualität* der Daten
- von der *Verfügbarkeit* der Daten
- vom *Preis* für den Erwerb der Daten

Quelldaten in verschiedenen Formaten

- (Relationale) Datenbanken
- Strukturierte Datenformate, z.B. XML, JSON
- Proprietäre Dateiformate
- Unstrukturierte Daten, z.B. Texte

Effizientes Laden von externen Daten ins DWH

- **Aufgabe:** Effizientestes Einbringen von externen Daten in DWH
- Ladevorgänge blockieren u.U. das Lesen aus dem DWH durch Schreibsperrern
- Wichtige Aspekte bezüglich der ETL-Performance
 - Integritätsbedingungen (Constraints) ausschalten?
 - Wann Indizes aktualisieren?
 - Isolationslevel für Transaktionen verändern?
 - ACID temporär lockern?



Satzbasiertes Laden

- Verwendung von Standard-Schnittstellen, z.B. JDBC
- Import im normalen Transaktionskontext des DBS
 - Isolationslevel senken, um Sperren zu vermeiden, bzw. parallel zu befüllen.
 - Sperren können durch häufiges **Commit** freigegeben werden.
- Trigger, Indizes und Constraints sind grundsätzlich aktiv und müssen manuell deaktiviert werden.
- Teilweise proprietäre Erweiterungen (z.B. Insert-Arrays) verfügbar.

Demo

Bulk-Load

- DBS-spezifische Erweiterungen zum Laden großer Datenmengen
- Import außerhalb des Transaktionskontext
- Keine Beachtung von Triggern oder Constraints
- Indizes werden erst nach Fertigstellung aktualisiert
- Sperre auf Ebene ganzer Tabelle anstatt auf Datensätzen
- Logging i.d.R. ausgeschaltet
- Parellelisierung möglich

Beispiel: MariaDB LOAD DATA

```
-- Load input file from tablespace directory  
LOAD DATA INFILE 'us-flights-2017.csv' INTO TABLE flights  
FIELDS TERMINATED BY ';' ENCLOSED BY '"' IGNORE 1 LINES;
```

```
FL_DATE;CARRIER;FL_NUM;ORIGIN;DEST;DEP_TIME;DEP_DELAY;TAXI_OUT;WHEELS_OFF;WHEELS_ON;TAXI_IN;ARR_TIME;ARR_DELAY;AIR_TIME;DISTANCE  
07.01.2017;AA;1766;Dallas/Fort Worth, TX;Las Vegas, NV;1051;-4.00;15.00;1106;1141;7.00;1148;-6.00;155.00;1055.00  
12.01.2017;DL;1712;Cleveland, OH;Atlanta, GA;948;73.00;15.00;1003;1130;5.00;1135;56.00;87.00;554.00  
18.01.2017;AA;2503;Los Angeles, CA;Chicago, IL;1122;-3.00;14.00;1136;1719;11.00;1730;-15.00;223.00;1744.00  
27.01.2017;DL;776;Phoenix, AZ;Los Angeles, CA;636;2.00;17.00;653;651;18.00;709;-1.00;58.00;370.00  
...
```